XRPL Validator — Daily Engineering Report · 2026-10-13

Period: 2026-10-13 00:00 to 24:00 US Eastern (2026-10-13 04:00 to 2026-10-14 04:00 UTC); ledgers #107,620,361 to #107,642,575. Every count from the ledger and the validator's logs stops at #107,642,575, closed 03:59:52 UTC. Generated 2026-10-21 00:47 EDT by report generator v3.0. Sources: the XRP Ledger (ledger headers and amendments, read from a reference rippled node), the validator's signing log and receipt logs, the deployment pipeline's gate, soak and health records, the git history of xrpl-validator and the XRPL Rust SDK, and data.xrpl.org. Scope: XRPL work only.

Revised on 2026-10-21 in report format v3; the original is kept in this repository's history. Built only from records that exist for this day. Available: ledger headers (the reference node holds ledgers from 2026-10-04 22:52 UTC); signing records; Step 1 receipt records (they begin 2026-08-29); Steps 2–5 receipt records (they begin 2026-10-09); deployment records (they begin 2026-09-11); xrpl-validator merge records (they begin 2026-07-04); amendment data; data.xrpl.org agreement scores. Not available: none.

Status

Impaired (all causes resolved or fixed) — 3 validations of a ledger hash the network did not validate.

Criteria. Normal: coverage outside planned maintenance at or above the 99.9% objective, every validation agreed with the network's validated ledger, and no engine receipt is open. Degraded: coverage below the objective, or an open engine receipt. Impaired: any validation that disagreed with the network, or coverage below 90%.

Executive summary

- **Validator:** signed 22,206 of 22,215 ledgers; 3 validations did NOT match the network's validated ledger. Unsigned: 9 during planned restarts.
- **Native engine:** 0 engine receipts recorded.
- **Changes:** 1 deployment (cycle 171), 47 commits merged into main.
- **Amendment fixCleanup3_4_0** holds majority; activation due ~2026-10-23 09:34 UTC.

Service levels

Measure	Value
Ledgers closed by the network	22,215
Validations issued	22,206
Coverage (validations / ledgers closed)	99.959%
Validations matching the network's validated ledger hash	22,203 of 22,206
Unsigned during planned restarts	9
Unsigned outside maintenance	0
Coverage outside planned maintenance (objective 99.9%)	100.000%
Restarts	1

Measure	Value
Basis: the hash the first peer announced as accepted (before cycle 183)	22,206
External agreement, data.xrpl.org (UTC day 2026-10-13)	0.99951 (10 missed of 20,413)

Restarts:

- 06:12:46 UTC: deployment of cycle 171; validations resumed at #107,622,436 (9 ledgers unsigned between the last validation before the restart and the first after it)

The basis rows sum to validations issued. Coverage counts the validations the validator issued for ledgers the network closed in the period. Agreement compares each validation's ledger hash with the network's validated hash for that ledger. data.xrpl.org scores UTC days from the validations its crawlers received, so it can differ.

Native engine conformance

Earlier FFI shadow log (libxrpl applied against the validator's mirror; superseded by Step 1): 129 transaction-level differences recorded this day, across 1 ledger. These were not triaged into receipts.

Ledgers with a receipt in the period (#107,620,361–#107,642,575), by step:

Step	Ledgers with a receipt
Step 1 · native engine	0
Step 2 · records	0
Step 3 · order	0
Step 4 · peer data	0
Step 4 · validations	0
Step 5 · own hash	0
Writer	0

Live counters are kept only for the current run; none exist for a past day.

Receipt register

No receipts this day.

Incidents

INC-01 · 3 validations of a ledger hash the network did not validate

Window	2026-10-13 11:35 UTC to 2026-10-14 00:41 UTC
Ledgers	#107,627,430, #107,639,526, #107,639,527
Impact	the validator signed a hash the network did not validate for these ledgers; data.xrpl.org scores such ledgers missed
Cause	Before cycle 183 the validator signed the ledger hash from the first peer that announced a ledger as accepted, without confirmation. For these ledgers that peer had built a different ledger, so the validator signed a hash the network never validated.

Action	commit d540e6e — fix(validator): sign a ledger only once the trusted validators reach quorum on its hash; deployed in cycle 183 at 2026-10-18 22:18 UTC

Change log

Deployments

Cycle	Main	Deployed	Test suites	Replay windows	Post-deploy health	Soak
171	`6c8ebc8`	06:13:35 UTC	1,987 passed / 0 failed; port 1,006 / 0	HIST60 60 ledgers, 0 mismatches, WIN98 98 ledgers, 0 mismatches	409 matched, 0 mismatched	#46 from 06:40 UTC: CLEAN (2026-10-14 06:42 UTC)

Cycle 171 delivered: `6c8ebc8` feat(node): independence Step 1 — the amendment guard, the signature drop count, a validator list that fails closed

Each deployment passes the unit and port test suites and replays mainnet history windows with zero mismatches before it ships, is health-checked after it starts, and then soaks for 24 hours.

Merged into main

Merged	Repo	Commit	Subject
15:39 UTC	xrpl-validator	`c984ec5`	docs: README — the native Rust engine is the production apply path since Oct 8; libxrpl is the cross-check; the independence plan and its status
15:40 UTC	xrpl-validator	`3c6194c`	docs: README rewritten for October 2026 — the native Rust engine writes the ledger state, what is still borrowed, the road to independence
15:59 UTC	xrpl-validator	`48d9897`	docs: README rewritten for October 2026 — the native Rust engine writes the ledger state, what is still borrowed, the road to independence
16:01 UTC	xrpl-validator	`5718682`	docs: README rewritten for October 2026 — the native Rust engine writes the ledger state, what is still borrowed, the road to independence
19:04 UTC	xrpl-validator	`beff740`	fix(node): the amendment guard's /metrics lines start at the margin (the first build's carried the source indentation)
19:04 UTC	xrpl-validator	`b83b414`	feat(tools): xrpl-watch's last-event line shows its age, and FIXED with the finding once its cause has shipped (receipts_fixed.json beside the writer log)
19:04 UTC	xrpl-validator	`493bc07`	Merge meta-2.2: independence Steps 2, 3 and 5 groundwork (own transaction records, canonical order and passes, fees, close time, the consensus round recorder) for cycle 172
19:04 UTC	xrpl-validator	`5d5e4b5`	fix(ledger): a line TrustSet deletes is erased without its reserve bits, as mainnet records it (Mainnet #107541200)
19:04 UTC	xrpl-validator	`f65d35d`	fix(ledger): a missing account is terNO_ACCOUNT, a retry, not "tefNO_ACCOUNT"
19:04 UTC	xrpl-validator	`5847f2c`	feat(node): the validation quorum, pure, for 5.2(b)
19:04 UTC	xrpl-validator	`49aa2fe`	fix(ledger): a payer short of the fee is tecINSUFF_FEE, charged what it holds
19:04 UTC	xrpl-validator	`c976229`	Merge branch 'record-5.2a' into meta-2.2
19:04 UTC	xrpl-validator	`677f7e1`	docs(node): 5.2(a) recorder — the measured disk figures
19:04 UTC	xrpl-validator	`731395f`	feat(node): independence 5.2(a) — shadow recorder for consensus rounds

Merged	Repo	Commit	Subject
19:04 UTC	xrpl-validator	`9be3a5b`	fix(probe): DX_META and DX_PASSES give the engine the transaction's hash, which the fixture's tx_json lacks and the AccountTxnID stamp reads
19:04 UTC	xrpl-validator	`152703f`	Merge branch 'audit-5.1' into meta-2.2
19:04 UTC	xrpl-validator	`473d2e4`	Merge branch 'closetime-5.3' into meta-2.2
19:04 UTC	xrpl-validator	`698bca3`	docs(consensus): independence 5.1 — audit of the March consensus code against rippled 3.4.1
19:04 UTC	xrpl-validator	`643f8ea`	docs: independence status 12 Oct — Step 2 built, 3.1/3.2/3.4/3.5 done offline, 5.3 in progress
19:04 UTC	xrpl-validator	`bf9999c`	fix(ledger): 5.3's cadence checks use is_multiple_of (clippy's deny-by-default modulo_one)
19:04 UTC	xrpl-validator	`d67b170`	feat(ledger): independence 5.3 — rippled's close time: resolution schedule, rounding, the no-consensus flag, the vote
19:04 UTC	xrpl-validator	`55b7b88`	feat(scripts): fetch_meta_ledgers.py --range, a run of consecutive headers (task 5.3)
19:04 UTC	xrpl-validator	`8a5973b`	feat(ledger): independence 3.5 — pseudo-transactions get Change::preflight/preclaim's checks
19:04 UTC	xrpl-validator	`76e622e`	fix(ledger): a trust line is deleted as updated, so its DeletedNode FinalFields carry the new balance and flags (Mainnet #107541106 F3DC8E3560FD)
19:04 UTC	xrpl-validator	`4d8231d`	feat(ledger): independence 3.4 — total_coins is the parent's less every fee charged
19:04 UTC	xrpl-validator	`f36632a`	fix(probe): run_passes takes the maps, not borrowing closures
19:04 UTC	xrpl-validator	`68cc5b0`	feat(scripts): fetch_meta_ledgers.py --parents, the header before each stored ledger (task 3.4)
19:04 UTC	xrpl-validator	`b63b2fd`	feat(probe): independence 3.2(b) — DX_PASSES rebuilds the ledger from its set: canonical order, apply passes, the engine's own results
19:04 UTC	xrpl-validator	`4c54dce`	Merge branch 'passes-3.2' into meta-2.2
19:04 UTC	xrpl-validator	`6b0bf25`	fix(ledger): 3.2(a) — a tecEXPIRED/tecKILLED from preclaim waits under tapRETRY like any tec
19:04 UTC	xrpl-validator	`c5aeb2d`	feat(ledger): independence 3.2(a) — rippled's apply passes, and an offline model of them
19:04 UTC	xrpl-validator	`c1fa280`	test(node): counters_accumulate tolerates the parallel tests sharing the process-wide counters
19:04 UTC	xrpl-validator	`ccb673e`	feat(node): independence 2.7 — the live shadow leg builds every record and checks our transaction tree against the header (XRPL_NATIVE_META=1)
19:04 UTC	xrpl-validator	`8150b47`	test(ledger): 3.1's mainnet check totals the runs and the results of later passes, for 3.2
19:04 UTC	xrpl-validator	`fa95e63`	test(ledger): 3.1's mainnet check reads UNLModify's empty Account as the zero account
19:04 UTC	xrpl-validator	`e1ce268`	feat(probe): TXTREE — the ledger's transaction tree from our own records, against the header
19:04 UTC	xrpl-validator	`56c2acf`	feat(ledger): independence 3.1 — rippled's canonical order, salted with the set's hash
19:04 UTC	xrpl-validator	`8e50f5d`	feat(ledger,node,probe): independence 2.4 Batch — the outer's own metadata and each applied inner's, ParentBatchID
19:04 UTC	xrpl-validator	`d4e2ca9`	test(ledger): name the row type in 2.2's node_changes test (clippy)

Merged	Repo	Commit	Subject
19:04 UTC	xrpl-validator	`0257b15`	fix(meta_check): TransactionResult goes to the codec as its number
19:04 UTC	xrpl-validator	`a2bf038`	feat(probe): independence 2.6 — DX_META compares our metadata with mainnet's, byte for byte
19:04 UTC	xrpl-validator	`b2fa569`	test(ledger): 2.3's canonical-form test borrows instead of cloning (clippy)
19:04 UTC	xrpl-validator	`2f5546f`	feat(ledger): independence 2.3 — the AffectedNodes builder
19:04 UTC	xrpl-validator	`6564950`	feat(ledger): independence 2.2 + 2.4 — each changed object's before and after, its threading, and the delivered amount
19:04 UTC	xrpl-validator	`524feca`	fix(scripts): the 2.1 check reads only mainnet's l*_expected.json fixtures
19:04 UTC	xrpl-validator	`48a92bf`	feat(scripts): independence 2.1's check against mainnet's records, and an s2 answer-key fetcher
19:04 UTC	xrpl-validator	`2524da0`	feat(ledger): independence 2.1 — the field metadata table, generated from rippled 3.4.1

Findings fixed

No engine findings merged into main this day.

Other engineering changes

- **fix(node): the amendment guard's /metrics lines start at the margin (the first build's carried the source indentation)** (`beff740` , xrpl-validator)
- **feat(tools): xrpl-watch's last-event line shows its age, and FIXED with the finding once its cause has shipped (receipts_fixed.json beside the writer log)** (`b83b414` , xrpl-validator)
- **fix(ledger): a line TrustSet deletes is erased without its reserve bits, as mainnet records it (Mainnet #107541200)** (`5d5e4b5` , xrpl-validator) rippled's reserve-clear blocks run before trustDelete (SetTrust.cpp:645-665): every reserved side with no interest has its lsf*Reserve bit cleared and sfFlags set, and on a delete that's every reserved side. The record of the erased line therefore shows Flags without them: #107541200's TrustSets show 0x200000 on mainnet, where ours (the write-before-delete of 76e622e) still showed 0x220000. The final state is …
- **fix(ledger): a missing account is terNO_ACCOUNT, a retry, not "tefNO_ACCOUNT"** (`f65d35d` , xrpl-validator) Live-path change. TxResult::NoAccount read "tefNO_ACCOUNT", which is not a rippled code. rippled 3.4.1's checkSeqProxy and checkFee return terNO_ACCOUNT (TER.h:213).
- **feat(node): the validation quorum, pure, for 5.2(b)** (`5847f2c` , xrpl-validator) quorum(unl, effective) mirrors rippled 3.4.1's ValidatorList::calculateQuorum formula (ValidatorList.cpp:1795-1796): max(ceil(0.8 x effective), ceil(0.6 x UNL)). - The arithmetic is single-precision float, as rippled's is. The 60% term rounds up at some multiples of 5: 45 x 0.6f is 27.000002, so a 45-list with 12 disabled needs 28 where exact maths says 27. - Today's 35 needs 28.
- **fix(ledger): a payer short of the fee is tecINSUFF_FEE, charged what it holds** (`49aa2fe` , xrpl-validator) Live-path change: native_apply's pipeline, which the native shadow and the native writer run.
- **feat(node): independence 5.2(a) — shadow recorder for consensus rounds** (`731395f` , xrpl-validator) Off unless XRPL_CONSENSUS_RECORD=1, and observe only: it decides nothing, drops nothing, and touches

neither signing nor the writer. The hot path is one bounded-channel try_send beside the S1/S2 checks (verified proposals and validations), the UNL load and the validator's ledger feed's header fetch. A writer thread does the rest; a full channel counts the event as lost.

- **fix(probe): DX_META and DX_PASSES give the engine the transaction's hash, which the fixture's tx_json lacks and the AccountTxnID stamp reads** (`9be3a5b`, xrpl-validator)
- **fix(ledger): 5.3's cadence checks use is_multiple_of (clippy's deny-by-default modulo_one)** (`bf9999c`, xrpl-validator) rippled's `ledgerSeq % kDecreaseLedgerTimeResolutionEvery == 0` with an every of 1 is always true; written with % it is clippy's modulo_one error.
- **feat(ledger): independence 5.3 — rippled's close time: resolution schedule, rounding, the no-consensus flag, the vote** (`d67b170`, xrpl-validator) ledger::close_time ports rippled 3.4.1's close-time rules: - next_resolution (getNextLedgerTimeResolution, LedgerTiming.h): resolutions 10, 20, 30, 60, 90, 120 s; after a close time not agreed, one step coarser on every ledger; after an agreed one, one step finer on a sequence divisible by 8. - round_close_time / effective_close_time (roundCloseTime, effCloseTime): to the nearest multiple, halves up, and at least …
- **feat(scripts): fetch_meta_ledgers.py --range, a run of consecutive headers (task 5.3)** (`55b7b88`, xrpl-validator) One binary `ledger` call with no transactions per ledger, at the same pacing and back-off, into DIR/headers/, kept when validated and its ledger_data hashes to its ledger_hash; the close-time check verifies the links between them.
- **feat(ledger): independence 3.5 — pseudo-transactions get Change::preflight/preclaim's checks** (`8a5973b`, xrpl-validator) rippled 3.4.1's Change transactor checks what our pseudo transactors did not: - preflight (with preflight0's part): no tfInnerBatchTxn; under LendingProtocol, no flag outside tfEnableAmendmentMask (temINVALID_FLAG); the zero account (temBAD_SRC_ACCOUNT, a new TxResult); a native zero fee (temBAD_FEE); no SigningPubKey, TxnSignature or Signers (temBAD_SIGNATURE); Sequence 0 and no PreviousTxnID (temBAD_SEQUENCE); - …
- **fix(ledger): a trust line is deleted as updated, so its DeletedNode FinalFields carry the new balance and flags (Mainnet #107541106 F3DC8E3560FD)** (`76e622e`, xrpl-validator) rippled updates the line's SLE and then erases that version: rippleCreditIOU sets the balance (zero) and clears the reserve flag before trustDelete, and TrustSet sets the new limit and flags before its trustDelete. The engine computed the updated line and deleted the stored one, so the deleted object's content at the deletion (task 2.2) was the old balance: our record of #107541106's payment showed the VOX line's …
- **feat(ledger): independence 3.4 — total_coins is the parent's less every fee charged** (`4d8231d`, xrpl-validator) header::total_coins_after(parent_total, fees_charged) is rippled's burn: an applied transaction, tes or tec, destroys the fee it charged at the end of Transactor::operator() (ctx_.destroyXRP -> Ledger::rawDestroyXRP, header_.drops -= fee, 3.4.1); nothing else destroys XRP. The charged fee is the Fee unless Transactor::reset capped a tec's to what its payer could pay; Batch inners carry Fee 0 and pseudo-transactions …
- **fix(probe): run_passes takes the maps, not borrowing closures** (`f36632a`, xrpl-validator)
- **feat(scripts): fetch_meta_ledgers.py --parents, the header before each stored ledger (task 3.4)** (`68cc5b0`, xrpl-validator) One binary `ledger` call with no transactions per stored ledger's parent, at the same pacing and back-off, into DIR/headers/, kept only when it is validated, its ledger_data hashes to its ledger_hash, and that hash is the stored ledger's parent_hash. Task 3.4 checks total_coins against it.
- **feat(probe): independence 3.2(b) — DX_PASSES rebuilds the ledger from its set: canonical order, apply passes, the engine's own results** (`b63b2fd`, xrpl-validator)

- **fix(ledger): 3.2(a) — a tecEXPIRED/tecKILLED from preclaim waits under tapRETRY like any tec** (`6b0bf25`, xrpl-validator) ALWAYS_CLAIMED holds only for the codes a transactor's doApply returns: preclaim's likelyToClaimFee is false under tapRETRY (applySteps.h), so a preclaim tec is not applied and is retried. The model now keeps the preclaim sources (NFTokenCreateOffer, OfferCreate, CheckCash and others' tecEXPIRED; LoanSet/LoanPay's tecKILLED) waiting; mainnet #106793245's NFTokenCreateOffer tecEXPIRED is one.
- **feat(ledger): independence 3.2(a) — rippled's apply passes, and an offline model of them** (`c5aeb2d`, xrpl-validator) close::apply_in_passes ports BuildLedger.cpp's applyTransactions (3.4.1): up to LEDGER_TOTAL_PASSES (3) over a canonical set; retryAssured (tapRETRY) while each pass changes something and through LEDGER_RETRY_PASSES (1); a pass that changes nothing ends the certain retries, and a claiming pass that changes nothing stops; what still retries after the last pass is left out. The caller's apply(tx, certain_retry) …
- **feat(node): independence 2.7 — the live shadow leg builds every record and checks our transaction tree against the header (XRPL_NATIVE_META=1)** (`ccb673e`, xrpl-validator)
- **feat(probe): TXTREE — the ledger's transaction tree from our own records, against the header** (`e1ce268`, xrpl-validator)
- **feat(ledger): independence 3.1 — rippled's canonical order, salted with the set's hash** (`56c2acf`, xrpl-validator) close::canonical_order was a plain sort (account, sequence, id). rippled 3.4.1's CanonicalTXSet (CanonicalTXSet.h/.cpp) orders a consensus set by the account copied into 32 bytes and XORed with a salt, then the SeqProxy (a sequence before a ticket, then by value: STTx::getSeqProxy), then the transaction id. The salt is the agreed set's own hash (RCLConsensus.cpp:502), the root of its SHAMap without metadata, which …
- **feat(ledger,node,probe): independence 2.4 Batch — the outer's own metadata and each applied inner's, ParentBatchID** (`8e50f5d`, xrpl-validator)
- **fix(meta_check): TransactionResult goes to the codec as its number** (`0257b15`, xrpl-validator)
- **feat(probe): independence 2.6 — DX_META compares our metadata with mainnet's, byte for byte** (`a2bf038`, xrpl-validator)
- **feat(ledger): independence 2.3 — the AffectedNodes builder** (`2f5546f`, xrpl-validator) tx_meta::affected_nodes(&[NodeChange]) -> Vec turns one transaction's changes into rippled's AffectedNodes JSON, in key order, by ApplyStateTable::apply's rules (ApplyStateTable.cpp:131-245, 3.4.1) and the field_meta flags: - NewFields: Create|Always fields that are not the default (each STI type's isDefault: an IOU or MPT zero never is, XRP zero is, a present account never is). - PreviousFields: the old …
- **feat(ledger): independence 2.2 + 2.4 — each changed object's before and after, its threading, and the delivered amount** (`6564950`, xrpl-validator) rippled builds a transaction's metadata (ApplyStateTable::apply) from two versions of every object it changed. For a deleted object the second version is its content at the deletion, not the base: an offer consumed and then deleted shows the consumed amounts in its DeletedNode FinalFields.
- **fix(scripts): the 2.1 check reads only mainnet's l*_expected.json fixtures** (`524feca`, xrpl-validator) Some fixture folders on the build box hold testnet and devnet campaign ledgers; the check is against mainnet's records, so a header with a network_id other than 0 is skipped and counted.
- **feat(scripts): independence 2.1's check against mainnet's records, and an s2 answer-key fetcher** (`48a92bf`, xrpl-validator) check_field_meta.py checks every field mainnet put in PreviousFields, FinalFields or NewFields against the generated table, by ApplyStateTable's rules; reports kSmdNever fields seen anywhere, Deleted

FinalFields / Created NewFields the table doesn't allow, and fields it doesn't know; and counts every (source, node type, section, field). It reads binary metadata (the txtree fixtures, s2 binary answers) and JSON (s2 …

- **feat(ledger): independence 2.1 — the field metadata table, generated from rippled 3.4.1** (`2524da0`, xrpl-validator) Every rippled field carries kSmd* flags that decide where it goes in a transaction's AffectedNodes (ApplyStateTable::apply): PreviousFields need ChangeOrig, a Modified node's FinalFields Always|ChangeNew, a Deleted node's FinalFields Always|DeleteFinal, a Created node's NewFields Create|Always. Nothing in our code knew them; Step 2's record builder needs them.
- **feat(node): independence 3.3 — the writer's own order and passes, in shadow (XRPL_NATIVE_ORDER=shadow|on, off by default)** (`a20a030`, xrpl-validator) Shadow applies each ledger in rippled's canonical order and retry passes first, keeps the order, the results and the bytes that gives, undoes it, and applies in the network's order as before; native_order counts and logs where the two differ (/api/engine native_order, /metrics, native_order.jsonl). On makes our order the apply, and a ledger it cannot reproduce a writer refusal. The per-transaction body moves into …
- **feat(node): 3.3's shadow checks total XRP live (task 3.4): the parent's total less the fees our walk burned, each the Fee capped at what its payer held, against the header's** (`6451e2a`, xrpl-validator) A receipt and a counter, never a refusal: the total is in the header, which the writer does not write.
- **fix(ledger): EscrowCancel refuses before CancelAfter and without one, whoever signs (A4)** (`7176e7c`, xrpl-validator) Live-path change.
- **fix(ledger): Clawback tests that the holder exists before the issuer's flags** (`dcee3e2`, xrpl-validator) Live-path change. It only bites when we build a ledger ourselves.
- **fix(ledger): NFTokenMint with Amount runs NFTokenCreateOffer's offer checks (A6)** (`22c2cf9`, xrpl-validator) Live-path change.
- **fix(ledger): PermissionedDomainSet refuses a missing credential issuer, tecNO_ISSUER (A5)** (`881a780`, xrpl-validator) Live-path change.
- **fix(node): checkPermission for a standalone delegated transaction (A2)** (`d1ffc5c`, xrpl-validator) Live-path change. It only bites when we build a ledger ourselves.
- **fix(ledger): AMMBid requires every AuthAccounts account to exist, terNO_ACCOUNT (A3)** (`9e3182f`, xrpl-validator) Live-path change. It only bites when we build a ledger ourselves.
- **fix(ledger): a missing AMM pool is terNO_AMM on a complete state (A1)** (`e7d153b`, xrpl-validator) Live-path change. It only bites when we build a ledger ourselves.
- **fix(node): A2 refuses a missing Delegate object only on a complete state** (`5a44553`, xrpl-validator) Live-path change.
- **feat(node): independence 4.1 — fetch a ledger from peers, proven to its ledger hash (shamap::wire, peer_fetch, peer_fetch_probe)** (`3d9f510`, xrpl-validator) shamap::wire reads rippled's SHAMap wire format (serializeForWire, SHAMapNodeID::getRawString) and assembles a ledger's transaction tree from fetched nodes, taking a node only at a position an accepted parent named and only when it hashes to what the parent holds there. peer_fetch::LedgerFetch drives rippled's own requests (liBASE for the header and the tree's root, then liTX_NODE three levels at a time) and …
- **fix(node): peer_fetch's result takes the header before the closure; Hash256 for the tests only** (`2343ee5`, xrpl-validator)
- **fix(node): peer_fetch_probe installs rustls' ring provider, as validation.rs does** (`5dd3455`, xrpl-validator)

- **feat(node): 4.1's live driver, off by default (XRPL_PEER_FETCH=1)** (`76670ef`, xrpl-validator) Each validated ledger the validator's ledger feed has proven against its header's transaction root (the tx-gate's pairs) is fetched from peers too and compared, byte for byte: peer_fetch::offer_network. The driver works one ledger at a time and spreads its requests: each peer at most once every 8 s, the longest-rested first, another one after a 6 s silence, and a peer that misses three replies (or sends a header that is not the …
- **feat(tools): xrpl-watch's INDEPENDENCE page (key i; 1 or Tab back): the road to a fully independent validator, live** (`9ac9d82`, xrpl-validator) The six steps of the plan as a road (LIVE / SHADOW / BUILT · OFF / STARTED / LATER), each with the running validator's own numbers: the amendment guard, signature checks and their enforcement, the validator list and whether it fails closed; our own records and trees (native_meta); our own order, results, bytes and total XRP (native_order); ledgers from peers (peer_fetch); the consensus round recorder and the signing …
- **feat(node): independence 4.5 — "validated" from our own count of trusted validations, in shadow (XRPL_OWN_VALIDATED=1, off by default)** (`fa66644`, xrpl-validator) own_validated counts, ledger by ledger, the signature-checked full validations from validators on the verified list (signing key to master through the list's manifests), less the negative UNL the newest written ledger carries, against rippled's quorum (consensus::quorum: max(ceil(0.8 x effective), ceil(0.6 x UNL))), as LedgerMaster::checkAccept does. Every ledger the validator's ledger feed takes because the reference node calls it validated is …
- **fix(node): 4.5 judges nothing before the validator list loads or for a ledger voted on before the tally listened; the TUI's step 4 shows our own "validated"** (`74287b2`, xrpl-validator) The first mainnet run's one short ledger was the one closing as the sessions connected.
- **feat(node): independence 5.5, first form — our own ledger hash, trailing the network (XRPL_OWN_HASH=1, off by default)** (`4f2e38e`, xrpl-validator) own_hash.rs assembles each landed ledger's header from what this validator made itself and hashes it (calculateLedgerHash): the state root from our hasher (ours when every byte written was our engine's, the skip list and protocol singletons included), the transaction root from our own records (ours when our order matched, 3.3), the total XRP as our previous total less our burn, the parent hash, resolution (5.3) and …
- **fix(tools): differential_probe's own-hash line reads rpc()'s answer as the result it already is** (`57068cd`, xrpl-validator)

Tests and campaigns

- **test(node): counters_accumulate tolerates the parallel tests sharing the process-wide counters** (`c1fa280`, xrpl-validator)
- **test(ledger): 3.1's mainnet check totals the runs and the results of later passes, for 3.2** (`8150b47`, xrpl-validator)
- **test(ledger): 3.1's mainnet check reads UNLModify's empty Account as the zero account** (`fa95e63`, xrpl-validator)
- **test(ledger): name the row type in 2.2's node_changes test (clippy)** (`d4e2ca9`, xrpl-validator)
- **test(ledger): 2.3's canonical-form test borrows instead of cloning (clippy)** (`b2fa569`, xrpl-validator)
- **test(node): differential_probe DX_SHADOW= — the live writer's propose() over a mainnet ledger's pre-state, in 3.3's shadow and on modes** (`1150110`, xrpl-validator)
- **test(node): DX_SHADOW builds the metadata key sets as the validator's ledger feed does (extract_affected_nodes, in TransactionIndex order)** (`a02ac0a`, xrpl-validator)

- **test(node): the 3.3 NoAccount audit's fixes, failing first** (`4774af3`, xrpl-validator)
- **test(ledger): two EscrowCancel tests now cancel past CancelAfter, as rippled requires** (`87e400c`, xrpl-validator)

Docs and operations

- **docs: README — the native Rust engine is the production apply path since Oct 8; libxrpl is the cross-check; the independence plan and its status** (`c984ec5`, xrpl-validator)
- **docs: README rewritten for October 2026 — the native Rust engine writes the ledger state, what is still borrowed, the road to independence** (`3c6194c`, xrpl-validator)
- **docs: README rewritten for October 2026 — the native Rust engine writes the ledger state, what is still borrowed, the road to independence** (`48d9897`, xrpl-validator)
- **docs: README rewritten for October 2026 — the native Rust engine writes the ledger state, what is still borrowed, the road to independence** (`5718682`, xrpl-validator)
- **Merge meta-2.2: independence Steps 2, 3 and 5 groundwork (own transaction records, canonical order and passes, fees, close time, the consensus round recorder) for cycle 172** (`493bc07`, xrpl-validator)
- **Merge branch 'record-5.2a' into meta-2.2** (`c976229`, xrpl-validator)
- **docs(node): 5.2(a) recorder — the measured disk figures** (`677f7e1`, xrpl-validator)
- **Merge branch 'audit-5.1' into meta-2.2** (`152703f`, xrpl-validator)
- **Merge branch 'closetime-5.3' into meta-2.2** (`473d2e4`, xrpl-validator)
- **docs(consensus): independence 5.1 — audit of the March consensus code against rippled 3.4.1** (`698bca3`, xrpl-validator)
- **docs: independence status 12 Oct — Step 2 built, 3.1/3.2/3.4/3.5 done offline, 5.3 in progress** (`643f8ea`, xrpl-validator)
- **Merge branch 'passes-3.2' into meta-2.2** (`4c54dce`, xrpl-validator)
- **Merge order-3.3: independence 3.3, the writer's own order and passes (XRPL_NATIVE_ORDER, off by default), total XRP live, DX_SHADOW** (`ecc2ac1`, xrpl-validator)
- **Merge noacct-fixes (5b): the 3.3 NoAccount audit's fixes — EscrowCancel's time gate, PermissionedDomainSet's issuer, NFTokenMint's offer checks, Clawback's order, AMMBid's AuthAccounts, checkPermission, terNO_AMM on a complete state** (`34cee5d`, xrpl-validator)
- **docs(independence): 5.5's first form — our own ledger hash, trailing the network (ships in cycle 173, James #49)** (`092ac3b`, xrpl-validator)

Independence program

No independence record for UTC day 2026-10-13; records begin 2026-10-20.

Network

Measure	Value
Ledgers closed	22,215 (#107,620,361–#107,642,575)

Measure	Value
Mean close interval	3.89 s
Longest close interval	10 s (before #107,631,112)
Closes at least 15 s apart	0
Empty ledgers	0

From ledger close times, which the network records at a 10-second resolution or coarser, so single intervals are approximate.

Amendment watch

Read from the XRP Ledger: the Amendments object as of the last ledger of the day, and the day's EnableAmendment pseudo-transactions.

Amendment	Majority since	Activation due
fixCleanup3_4_0	2026-10-09 09:34 UTC	2026-10-23 09:34 UTC

Amendments enabled on mainnet: 97.

Open items

- Amendment fixCleanup3_4_0: activation due ~2026-10-23 09:34 UTC.

Corrections to earlier reports

None.

Appendix — commits

Committed	Merged into main	Repo	Commit	Subject
10-13 15:39	10-13 15:39	xrpl-validator	`c984ec5`	docs: README — the native Rust engine is the production apply path since Oct 8; libxrpl is the cross-check; the independence plan and its status
10-13 15:40	10-13 15:40	xrpl-validator	`3c6194c`	docs: README rewritten for October 2026 — the native Rust engine writes the ledger state, what is still borrowed, the road to independence
10-13 15:59	10-13 15:59	xrpl-validator	`48d9897`	docs: README rewritten for October 2026 — the native Rust engine writes the ledger state, what is still borrowed, the road to independence
10-13 16:01	10-13 16:01	xrpl-validator	`5718682`	docs: README rewritten for October 2026 — the native Rust engine writes the ledger state, what is still borrowed, the road to independence
10-13 06:42	10-13 19:04	xrpl-validator	`beff740`	fix(node): the amendment guard's /metrics lines start at the margin (the first build's carried the source indentation)
10-13 15:12	10-13 19:04	xrpl-validator	`b83b414`	feat(tools): xrpl-watch's last-event line shows its age, and FIXED with the finding once its cause has shipped (receipts_fixed.json beside the writer log)
10-13 19:04	10-13 19:04	xrpl-validator	`493bc07`	Merge meta-2.2: independence Steps 2, 3 and 5 groundwork (own transaction records, canonical order and passes, fees, close time, the consensus round recorder) for cycle 172

Committed	Merged into main	Repo	Commit	Subject
10-12 19:24	10-13 19:04	xrpl-validator	`5d5e4b5`	fix(ledger): a line TrustSet deletes is erased without its reserve bits, as mainnet records it (Mainnet #107541200)
10-12 19:15	10-13 19:04	xrpl-validator	`f65d35d`	fix(ledger): a missing account is terNO_ACCOUNT, a retry, not "tefNO_ACCOUNT"
10-12 19:06	10-13 19:04	xrpl-validator	`5847f2c`	feat(node): the validation quorum, pure, for 5.2(b)
10-12 19:06	10-13 19:04	xrpl-validator	`49aa2fe`	fix(ledger): a payer short of the fee is tecINSUFF_FEE, charged what it holds
10-12 18:52	10-13 19:04	xrpl-validator	`c976229`	Merge branch 'record-5.2a' into meta-2.2
10-12 18:51	10-13 19:04	xrpl-validator	`677f7e1`	docs(node): 5.2(a) recorder — the measured disk figures
10-12 18:44	10-13 19:04	xrpl-validator	`731395f`	feat(node): independence 5.2(a) — shadow recorder for consensus rounds
10-12 18:24	10-13 19:04	xrpl-validator	`9be3a5b`	fix(probe): DX_META and DX_PASSES give the engine the transaction's hash, which the fixture's tx_json lacks and the AccountTxnID stamp reads
10-12 18:22	10-13 19:04	xrpl-validator	`152703f`	Merge branch 'audit-5.1' into meta-2.2
10-12 18:22	10-13 19:04	xrpl-validator	`473d2e4`	Merge branch 'closetime-5.3' into meta-2.2
10-12 18:22	10-13 19:04	xrpl-validator	`698bca3`	docs(consensus): independence 5.1 — audit of the March consensus code against rippled 3.4.1
10-12 16:58	10-13 19:04	xrpl-validator	`643f8ea`	docs: independence status 12 Oct — Step 2 built, 3.1/3.2/3.4/3.5 done offline, 5.3 in progress
10-12 16:46	10-13 19:04	xrpl-validator	`bf9999c`	fix(ledger): 5.3's cadence checks use is_multiple_of (clippy's deny-by-default modulo_one)
10-12 16:45	10-13 19:04	xrpl-validator	`d67b170`	feat(ledger): independence 5.3 — rippled's close time: resolution schedule, rounding, the no-consensus flag, the vote
10-12 16:42	10-13 19:04	xrpl-validator	`55b7b88`	feat(scripts): fetch_meta_ledgers.py --range, a run of consecutive headers (task 5.3)
10-12 16:31	10-13 19:04	xrpl-validator	`8a5973b`	feat(ledger): independence 3.5 — pseudo-transactions get Change::preflight/preclaim's checks
10-12 16:28	10-13 19:04	xrpl-validator	`76e622e`	fix(ledger): a trust line is deleted as updated, so its DeletedNode FinalFields carry the new balance and flags (Mainnet #107541106 F3DC8E3560FD)
10-12 16:17	10-13 19:04	xrpl-validator	`4d8231d`	feat(ledger): independence 3.4 — total_coins is the parent's less every fee charged
10-12 16:15	10-13 19:04	xrpl-validator	`f36632a`	fix(probe): run_passes takes the maps, not borrowing closures
10-12 16:14	10-13 19:04	xrpl-validator	`68cc5b0`	feat(scripts): fetch_meta_ledgers.py --parents, the header before each stored ledger (task 3.4)
10-12 16:14	10-13 19:04	xrpl-validator	`b63b2fd`	feat(probe): independence 3.2(b) — DX_PASSES rebuilds the ledger from its set: canonical order, apply passes, the engine's own results
10-12 16:12	10-13 19:04	xrpl-validator	`4c54dce`	Merge branch 'passes-3.2' into meta-2.2
10-12 16:00	10-13 19:04	xrpl-validator	`6b0bf25`	fix(ledger): 3.2(a) — a tecEXPIRED/tecKILLED from preclaim waits under tapRETRY like any tec
10-12 15:47	10-13 19:04	xrpl-validator	`c5aeb2d`	feat(ledger): independence 3.2(a) — rippled's apply passes, and an offline model of them
10-12 15:46	10-13 19:04	xrpl-validator	`c1fa280`	test(node): counters_accumulate tolerates the parallel tests sharing the process-wide counters

Committed	Merged into main	Repo	Commit	Subject
10-12 15:44	10-13 19:04	xrpl-validator	`ccb673e`	feat(node): independence 2.7 — the live shadow leg builds every record and checks our transaction tree against the header (XRPL_NATIVE_META=1)
10-12 15:40	10-13 19:04	xrpl-validator	`8150b47`	test(ledger): 3.1's mainnet check totals the runs and the results of later passes, for 3.2
10-12 15:36	10-13 19:04	xrpl-validator	`fa95e63`	test(ledger): 3.1's mainnet check reads UNLModify's empty Account as the zero account
10-12 15:36	10-13 19:04	xrpl-validator	`e1ce268`	feat(probe): TXTREE — the ledger's transaction tree from our own records, against the header
10-12 15:30	10-13 19:04	xrpl-validator	`56c2acf`	feat(ledger): independence 3.1 — rippled's canonical order, salted with the set's hash
10-12 15:30	10-13 19:04	xrpl-validator	`8e50f5d`	feat(ledger,node,probe): independence 2.4 Batch — the outer's own metadata and each applied inner's, ParentBatchID
10-12 15:00	10-13 19:04	xrpl-validator	`d4e2ca9`	test(ledger): name the row type in 2.2's node_changes test (clippy)
10-12 15:00	10-13 19:04	xrpl-validator	`0257b15`	fix(meta_check): TransactionResult goes to the codec as its number
10-12 15:00	10-13 19:04	xrpl-validator	`a2bf038`	feat(probe): independence 2.6 — DX_META compares our metadata with mainnet's, byte for byte
10-12 14:27	10-13 19:04	xrpl-validator	`b2fa569`	test(ledger): 2.3's canonical-form test borrows instead of cloning (clippy)
10-12 14:24	10-13 19:04	xrpl-validator	`2f5546f`	feat(ledger): independence 2.3 — the AffectedNodes builder
10-12 14:18	10-13 19:04	xrpl-validator	`6564950`	feat(ledger): independence 2.2 + 2.4 — each changed object's before and after, its threading, and the delivered amount
10-12 13:43	10-13 19:04	xrpl-validator	`524feca`	fix(scripts): the 2.1 check reads only mainnet's l*_expected.json fixtures
10-12 13:41	10-13 19:04	xrpl-validator	`48a92bf`	feat(scripts): independence 2.1's check against mainnet's records, and an s2 answer-key fetcher
10-12 13:41	10-13 19:04	xrpl-validator	`2524da0`	feat(ledger): independence 2.1 — the field metadata table, generated from rippled 3.4.1
10-13 19:17	—	xrpl-validator	`a20a030`	feat(node): independence 3.3 — the writer's own order and passes, in shadow (XRPL_NATIVE_ORDER=shadow|on, off by default)
10-13 19:31	—	xrpl-validator	`6451e2a`	feat(node): 3.3's shadow checks total XRP live (task 3.4): the parent's total less the fees our walk burned, each the Fee capped at what its payer held, against the header's
10-13 19:33	—	xrpl-validator	`1150110`	test(node): differential_probe DX_SHADOW= — the live writer's propose() over a mainnet ledger's pre-state, in 3.3's shadow and on modes
10-13 19:46	—	xrpl-validator	`a02ac0a`	test(node): DX_SHADOW builds the metadata key sets as the validator's ledger feed does (extract_affected_nodes, in TransactionIndex order)
10-13 20:14	—	xrpl-validator	`7176e7c`	fix(ledger): EscrowCancel refuses before CancelAfter and without one, whoever signs (A4)
10-13 20:14	—	xrpl-validator	`4774af3`	test(node): the 3.3 NoAccount audit's fixes, failing first
10-13 20:15	—	xrpl-validator	`dcee3e2`	fix(ledger): Clawback tests that the holder exists before the issuer's flags
10-13 20:15	—	xrpl-validator	`22c2cf9`	fix(ledger): NFTokenMint with Amount runs NFTokenCreateOffer's offer checks (A6)
10-13 20:15	—	xrpl-validator	`881a780`	fix(ledger): PermissionedDomainSet refuses a missing credential issuer, tecNO_ISSUER (A5)
10-13 20:15	—	xrpl-validator	`d1ffc5c`	fix(node): checkPermission for a standalone delegated transaction (A2)

Committed	Merged into main	Repo	Commit	Subject
10-13 20:15	—	xrpl-validator	`9e3182f`	fix(ledger): AMMBid requires every AuthAccounts account to exist, terNO_ACCOUNT (A3)
10-13 20:18	—	xrpl-validator	`e7d153b`	fix(ledger): a missing AMM pool is terNO_AMM on a complete state (A1)
10-13 20:31	—	xrpl-validator	`5a44553`	fix(node): A2 refuses a missing Delegate object only on a complete state
10-13 20:31	—	xrpl-validator	`87e400c`	test(ledger): two EscrowCancel tests now cancel past CancelAfter, as rippled requires
10-13 20:41	—	xrpl-validator	`ecc2ac1`	Merge order-3.3: independence 3.3, the writer's own order and passes (XRPL_NATIVE_ORDER, off by default), total XRP live, DX_SHADOW
10-13 20:41	—	xrpl-validator	`34cee5d`	Merge noacct-fixes (5b): the 3.3 NoAccount audit's fixes — EscrowCancel's time gate, PermissionedDomainSet's issuer, NFTokenMint's offer checks, Clawback's order, AMMBid's AuthAccounts, checkPermission, terNO_AMM on a complete state
10-13 21:26	—	xrpl-validator	`3d9f510`	feat(node): independence 4.1 — fetch a ledger from peers, proven to its ledger hash (shamap::wire, peer_fetch, peer_fetch_probe)
10-13 21:27	—	xrpl-validator	`2343ee5`	fix(node): peer_fetch's result takes the header before the closure; Hash256 for the tests only
10-13 21:28	—	xrpl-validator	`5dd3455`	fix(node): peer_fetch_probe installs rustls' ring provider, as validation.rs does
10-13 21:34	—	xrpl-validator	`76670ef`	feat(node): 4.1's live driver, off by default (XRPL_PEER_FETCH=1)
10-13 21:52	—	xrpl-validator	`9ac9d82`	feat(tools): xrpl-watch's INDEPENDENCE page (key i; 1 or Tab back): the road to a fully independent validator, live
10-14 00:41	—	xrpl-validator	`fa66644`	feat(node): independence 4.5 — "validated" from our own count of trusted validations, in shadow (XRPL_OWN_VALIDATED=1, off by default)
10-14 00:52	—	xrpl-validator	`74287b2`	fix(node): 4.5 judges nothing before the validator list loads or for a ledger voted on before the tally listened; the TUI's step 4 shows our own "validated"
10-14 01:42	—	xrpl-validator	`4f2e38e`	feat(node): independence 5.5, first form — our own ledger hash, trailing the network (XRPL_OWN_HASH=1, off by default)
10-14 01:50	—	xrpl-validator	`57068cd`	fix(tools): differential_probe's own-hash line reads rpc()'s answer as the result it already is
10-14 02:13	—	xrpl-validator	`092ac3b`	docs(independence): 5.5's first form — our own ledger hash, trailing the network (ships in cycle 173, James #49)

Times in UTC. A commit appears on the day it was committed to main or merged into it.